PRÁCTICA 01

Fundamentos de Python

Soluciones completas de los ejercicios

Tema	Sintaxis, control de flujo, funciones, colecciones, Pandas, POO y pruebas
Lenguaje	Python 3.14
Dependencias	pandas, matplotlib
Modalidad	Individual
Fecha	Octubre de 2026
Entrega	Código fuente, documento de soluciones y salidas generadas

Documento de soluciones con explicación, código fuente y evidencia de ejecución.

Contenido

- 1. Presentación
- 2. Sintaxis básica
- 3. Estructuras de control
- 4. Funciones
- 5. Colecciones
- 6. Módulos y paquetes
- 7. Pandas y manipulación de datos
- 8. Visualización de datos
- 9. Programación orientada a objetos
- 10. Manejo de errores y excepciones
- 11. Manejo de archivos
- 12. Programación funcional
- 13. Programación asíncrona
- 14. Testing con unittest
- 15. Conclusiones
- Anexo A. Código completo
- Anexo B. Ejecución en Visual Studio y VS Code

1. Presentación

Este documento reúne la resolución de todos los ejercicios propuestos en la práctica de Fundamentos de Python. Cada ejercicio incluye el enunciado, la explicación de la solución, el código fuente y una salida representativa obtenida al ejecutar el programa.

El código se organiza en funciones y clases con nombres descriptivos, validación de entradas y un menú interactivo. No contiene comentarios ni marcas de edición: está preparado para abrirse, ejecutarse y depurarse directamente en Visual Studio o Visual Studio Code.

Elemento	Detalle
Archivo principal	soluciones_fundamentos.py
Módulo auxiliar	utilidades.py
Pruebas unitarias	test_soluciones.py
Dependencias	pandas, matplotlib
Salidas	salidas/estudiantes.csv, salidas/nombres.txt, salidas/graficos/
Ejecución rápida	py soluciones_fundamentos.py --demo

La demostración completa se ejecuta con el argumento --demo. Sin argumentos, el programa muestra un menú con veinte opciones interactivas que agrupan los ejercicios. Las pruebas se ejecutan con py -m unittest -v.

> Criterio de entrega: cada solución es ejecutable, valida sus entradas, separa la lógica de la interfaz y genera evidencia verificable de su funcionamiento.

2. Sintaxis básica

Los primeros ejercicios refuerzan la declaración de variables, las operaciones aritméticas y la salida formateada. Las funciones reciben los valores como parámetros para poder probarlas de forma aislada.

2.1 Promedio de tres números

ENUNCIADO

Escribe un programa que calcule el promedio de tres números ingresados por el usuario.

SOLUCIÓN

Se solicitan tres valores numéricos, se suman y se divide el total entre tres. El resultado se presenta con dos decimales.

CÓDIGO

```
def promedio_tres_numeros(a: float, b: float, c: float) -> float:
    return (a + b + c) / 3
```

SALIDA

```
Promedio: 9.00
```

2.2 Conversión de Celsius a Fahrenheit

ENUNCIADO

Crea un programa que convierta grados Celsius a Fahrenheit.

SOLUCIÓN

Se aplica la fórmula F = C × 9/5 + 32. La función devuelve un valor decimal y el programa lo muestra con dos decimales.

CÓDIGO

```
def celsius_a_fahrenheit(celsius: float) -> float:
    return celsius * 9 / 5 + 32
```

SALIDA

```
25 °C = 77.00 °F
```

2.3 Área de un triángulo

ENUNCIADO

Programa que calcule el área de un triángulo.

SOLUCIÓN

El área se obtiene multiplicando la base por la altura y dividiendo entre dos. La función concentra la fórmula y el menú se encarga de solicitar los datos.

CÓDIGO

```
def area_triangulo(base: float, altura: float) -> float:
    return base * altura / 2
```

SALIDA

```
Área: 30.00
```

3. Estructuras de control

Se utilizan condicionales, bucles y estructuras de selección para resolver problemas de decisión y repetición. La calculadora incorpora un ciclo que se mantiene activo hasta que el usuario elige salir.

3.1 Validación para votar

ENUNCIADO

Programa que determine si una persona puede votar según su edad.

SOLUCIÓN

Se valida que la edad no sea negativa y se compara con el umbral de 18 años. La función devuelve un valor booleano y el programa muestra el mensaje adecuado.

CÓDIGO

```
def puede_votar(edad: int) -> bool:
    if edad < 0:
        raise ValueError("La edad no puede ser negativa.")
    return edad >= 18
```

SALIDA

```
Edad 16: no puede votar
Edad 18: puede votar
Edad 30: puede votar
```

3.2 Tabla de multiplicar

ENUNCIADO

Crea un programa que muestre la tabla de multiplicar de un número.

SOLUCIÓN

La función genera una lista de cadenas con los productos del 1 al 10. El menú solicita el número y recorre la lista para imprimir la tabla completa.

CÓDIGO

```
def tabla_multiplicar(numero: int, limite: int = 10) -> list[str]:
    return [f"{numero} x {valor} = {numero * valor}" for valor in range(1, limite + 1)]
```

SALIDA

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

3.3 Menú de calculadora

ENUNCIADO

Programa que simule un menú con las opciones 1. Sumar, 2. Restar y 3. Salir.

SOLUCIÓN

El ciclo while muestra el menú, valida la opción y solicita dos números. La operación se selecciona con una estructura condicional y el programa termina cuando el usuario escribe 3.

CÓDIGO

```
def menu_calculadora() -> None:
    while True:
        print("\n1. Sumar")
        print("2. Restar")
        print("3. Salir")
        opcion = input("Seleccione una opcion: ").strip()
        if opcion == "3":
            print("Calculadora finalizada.")
            return
        if opcion not in {"1", "2"}:
            print("Opcion invalida.")
            continue
        primer_numero = solicitar_float("Primer numero: ")
        segundo_numero = solicitar_float("Segundo numero: ")
        resultado = (
            primer_numero + segundo_numero
            if opcion == "1"
            else primer_numero - segundo_numero
        )
        print(f"Resultado: {resultado:.2f}")
```

SALIDA

```
1. Sumar
2. Restar
3. Salir
Resultado: 13.00

1. Sumar
2. Restar
3. Salir
Resultado: 3.00

1. Sumar
2. Restar
3. Salir
Calculadora finalizada.
```

4. Funciones

Las funciones permiten encapsular cálculos y reutilizarlos. En esta sección se trabajan acumuladores, divisibilidad y recorridos de colecciones.

4.1 Factorial de un número

ENUNCIADO

Crea una función que calcule el factorial de un número.

SOLUCIÓN

Se valida que el número no sea negativo y se utiliza un acumulador multiplicativo desde 2 hasta el número indicado. El caso 0! se resuelve con el valor inicial 1.

CÓDIGO

```
def factorial(numero: int) -> int:
    if numero < 0:
        raise ValueError("El factorial no esta definido para numeros negativos.")
    resultado = 1
    for valor in range(2, numero + 1):
        resultado *= valor
    return resultado
```

SALIDA

```
0! = 1
5! = 120
7! = 5040
```

4.2 Verificación de número primo

ENUNCIADO

Crea una función que verifique si un número es primo.

SOLUCIÓN

Se descartan los números menores que 2 y los pares. Después se prueban los divisores impares hasta la raíz cuadrada del número, lo que reduce el costo del algoritmo.

CÓDIGO

```
def es_primo(numero: int) -> bool:
    if numero < 2:
        return False
    if numero == 2:
        return True
    if numero % 2 == 0:
        return False
    limite = math.isqrt(numero)
    for divisor in range(3, limite + 1, 2):
        if numero % divisor == 0:
            return False
    return True
```

SALIDA

```
17 es primo: True
21 es primo: False
```

4.3 Mayor de una lista

ENUNCIADO

Crea una función que reciba una lista y devuelva el mayor número.

SOLUCIÓN

La función valida que la lista no esté vacía y utiliza max para obtener el mayor elemento. El menú permite ingresar los números separados por comas.

CÓDIGO

```
def mayor_lista(numeros: Sequence[float]) -> float:
    if not numeros:
        raise ValueError("La lista no puede estar vacia.")
    return max(numeros)
```

SALIDA

```
Mayor: 45
```

5. Colecciones

Se emplean diccionarios, listas y conjuntos para organizar información, ordenar datos y eliminar elementos repetidos.

5.1 Personas mayores de edad

ENUNCIADO

Crea un diccionario con nombres y edades e imprime solo los mayores de edad.

SOLUCIÓN

Se recorre el diccionario y se conservan las entradas cuya edad sea mayor o igual a 18. La comprensión de diccionario devuelve un nuevo diccionario filtrado.

CÓDIGO

```
def mayores_de_edad(personas: dict[str, int]) -> dict[str, int]:
    return {
        nombre: edad
        for nombre, edad in personas.items()
        if edad >= 18
    }
```

SALIDA

```
{'Luis': 25, 'Maria': 30}
```

5.2 Ordenamiento de una lista

ENUNCIADO

Crea una lista de números y ordénala de menor a mayor.

SOLUCIÓN

La función recibe cualquier iterable y devuelve una lista nueva ordenada. No modifica la colección original, lo que evita efectos secundarios.

CÓDIGO

```
def ordenar_numeros(numeros: Iterable[float]) -> list[float]:
    return sorted(numeros)
```

SALIDA

```
[1, 3, 5, 7, 9]
```

5.3 Eliminación de duplicados

ENUNCIADO

Usa un conjunto para eliminar duplicados de una lista.

SOLUCIÓN

La función convierte la lista en un conjunto, lo que elimina automáticamente los valores repetidos. Para presentar el resultado de forma legible se ordena el conjunto antes de imprimirlo.

CÓDIGO

```
def eliminar_duplicados(elementos: Iterable[object]) -> set[object]:
    return set(elementos)
```

SALIDA

```
[1, 2, 3, 4, 5]
```

6. Módulos y paquetes

6.1 Módulo utilidades

ENUNCIADO

Crea un módulo llamado utilidades.py con una función que convierta millas a kilómetros e impórtalo.

SOLUCIÓN

El módulo contiene una función independiente que aplica el factor de conversión 1.609344. El archivo principal la importa y la utiliza sin duplicar la lógica.

CÓDIGO

```
def millas_a_kilometros(millas: float) -> float:
    return millas * 1.609344
```

SALIDA

```
10 millas = 16.09 kilómetros
```

El archivo utilidades.py se encuentra en la misma carpeta que el programa principal. La importación funciona al ejecutar desde esa carpeta o al abrir el proyecto completo en Visual Studio.

CÓDIGO DEL MÓDULO UTILIDADES.PY

```
def millas_a_kilometros(millas: float) -> float:
    return millas * 1.609344
```

7. Pandas y manipulación de datos

Se construye un DataFrame con estudiantes y tres notas. A partir de él se calcula el promedio, se filtran los estudiantes destacados y se agrega una columna con el estado académico.

7.1 DataFrame de estudiantes y promedio

ENUNCIADO

Crea un DataFrame con nombres de estudiantes y notas, y calcula el promedio.

SOLUCIÓN

Se crean tres columnas de notas y se calcula el promedio por fila con mean(axis=1). El resultado se redondea a dos decimales para facilitar la lectura.

CÓDIGO

```
def crear_dataframe_estudiantes() -> pd.DataFrame:
    datos = {
        "Estudiante": ["Ana", "Luis", "Maria", "Carlos", "Sofia", "Diego"],
        "Nota 1": [85, 62, 91, 74, 58, 88],
        "Nota 2": [90, 70, 87, 68, 65, 92],
        "Nota 3": [78, 55, 95, 80, 72, 85],
    }
    dataframe = pd.DataFrame(datos)
    dataframe["Promedio"] = (
        dataframe[["Nota 1", "Nota 2", "Nota 3"]]
        .mean(axis=1)
        .round(2)
    )
    return dataframe
```

SALIDA

```
Estudiante  Nota 1  Nota 2  Nota 3  Promedio
       Ana      85      90      78     84.33
      Luis      62      70      55     62.33
     Maria      91      87      95     91.00
    Carlos      74      68      80     74.00
     Sofia      58      65      72     65.00
     Diego      88      92      85     88.33
Promedio general: 77.50
```

TABLA GENERADA

Estudiante	Nota 1	Nota 2	Nota 3	Promedio
Ana	85	90	78	84.33
Luis	62	70	55	62.33
Maria	91	87	95	91.00
Carlos	74	68	80	74.00
Sofia	58	65	72	65.00
Diego	88	92	85	88.33

7.2 Filtro de estudiantes con promedio mayor a 70

ENUNCIADO

Filtra los estudiantes que tengan nota mayor a 70.

SOLUCIÓN

Se aplica una condición booleana sobre la columna Promedio. El filtro utiliza el operador mayor que, por lo que un promedio exactamente igual a 70 no se incluye.

CÓDIGO

```
def filtrar_estudiantes_aprobados(
    dataframe: pd.DataFrame, umbral: float = 70
) -> pd.DataFrame:
    return dataframe[dataframe["Promedio"] > umbral].copy()
```

SALIDA

```
Estudiante  Promedio
       Ana     84.33
     Maria     91.00
    Carlos     74.00
     Diego     88.33
```

7.3 Columna de estado académico

ENUNCIADO

Agrega una nueva columna que indique si el estudiante aprobó o no.

SOLUCIÓN

Se compara el promedio con el umbral de 70 y se asigna Aprobado o Reprobado. La función trabaja sobre una copia para no alterar el DataFrame original.

CÓDIGO

```
def agregar_estado(dataframe: pd.DataFrame, umbral: float = 70) -> pd.DataFrame:
    resultado = dataframe.copy()
    resultado["Estado"] = resultado["Promedio"].apply(
        lambda nota: "Aprobado" if nota >= umbral else "Reprobado"
    )
    return resultado
```

SALIDA

```
Estudiante  Nota 1  Nota 2  Nota 3  Promedio    Estado
       Ana      85      90      78     84.33  Aprobado
      Luis      62      70      55     62.33 Reprobado
     Maria      91      87      95     91.00  Aprobado
    Carlos      74      68      80     74.00  Aprobado
     Sofia      58      65      72     65.00 Reprobado
     Diego      88      92      85     88.33  Aprobado
```

8. Visualización de datos

Los gráficos se generan con matplotlib y se guardan en la carpeta salidas/graficos. Se utilizan títulos, etiquetas, colores y valores sobre las barras para que las figuras puedan presentarse sin explicación adicional.

8.1 Diagrama de barras de ventas mensuales

ENUNCIADO

Grafica un diagrama de barras con las ventas mensuales de un negocio.

SOLUCIÓN

Se comparan seis meses mediante barras verticales. Las etiquetas de datos y la cuadrícula horizontal facilitan la lectura de las cifras.

CÓDIGO

```
def grafico_barras_ventas() -> Path:
    meses = ["Ene", "Feb", "Mar", "Abr", "May", "Jun"]
    ventas = [1200, 1500, 1800, 1650, 2100, 2400]
    figura, eje = plt.subplots(figsize=(8, 4.8))
    barras = eje.bar(meses, ventas, color="#2563EB", edgecolor="#1E3A8A")
    eje.set_title("Ventas mensuales del negocio")
    eje.set_xlabel("Mes")
    eje.set_ylabel("Ventas (USD)")
    eje.grid(axis="y", linestyle="--", alpha=0.4)
    for lado in ("top", "right"):
        eje.spines[lado].set_visible(False)
    eje.bar_label(barras, fmt="%d", padding=3)
    return guardar_figura(figura, "ventas_mensuales.png")
```

SALIDA

```
Gráfico generado: salidas/graficos/ventas_mensuales.png
```

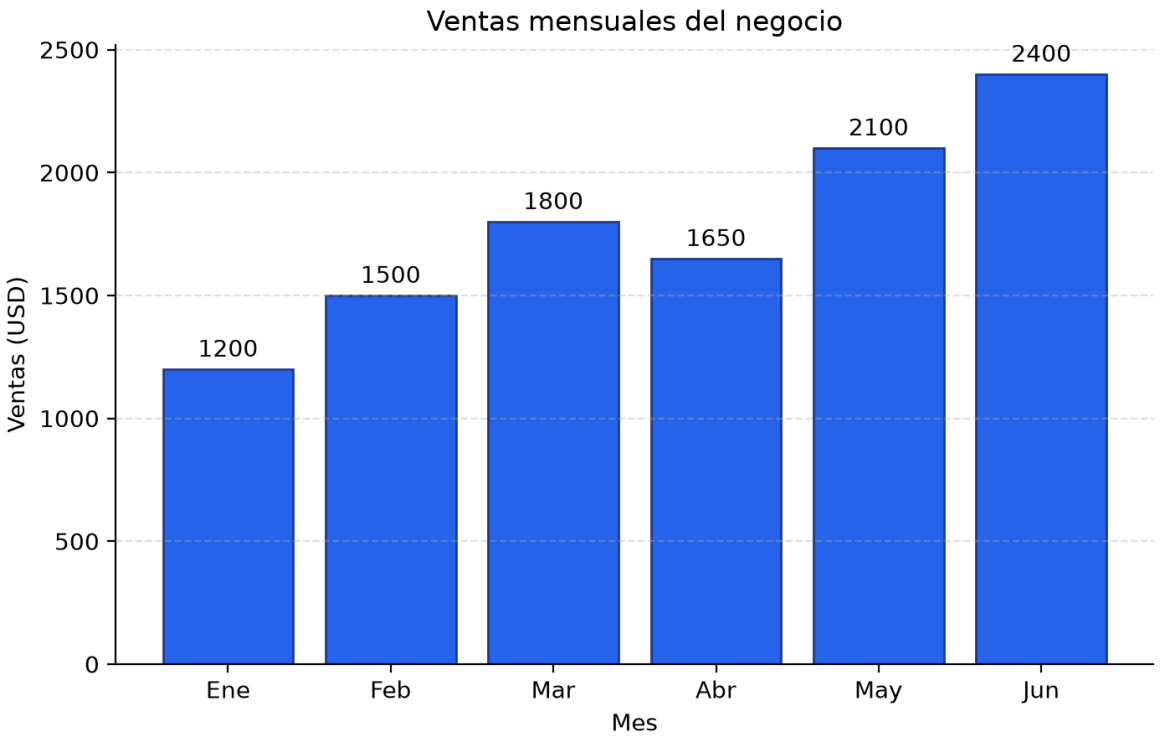


Figura 1. Ventas mensuales del negocio.

8.2 Diagrama de líneas de temperatura

ENUNCIADO

Grafica un diagrama de líneas con la temperatura promedio de 7 días.

SOLUCIÓN

La línea continua muestra la evolución diaria y el área sombreada refuerza la tendencia. Los marcadores identifican cada observación.

CÓDIGO

```
def grafico_lineas_temperatura() -> Path:
    dias = ["Lun", "Mar", "Mie", "Jue", "Vie", "Sab", "Dom"]
    temperaturas = [22, 24, 23, 25, 27, 26, 24]
    figura, eje = plt.subplots(figsize=(8, 4.8))
    eje.plot(
        dias,
        temperaturas,
        marker="o",
        markersize=7,
        linewidth=2.5,
        color="#DC2626",
    )
    eje.fill_between(dias, temperaturas, color="#DC2626", alpha=0.12)
    eje.set_title("Temperatura promedio durante siete dias")
    eje.set_xlabel("Dia")
    eje.set_ylabel("Temperatura (C)")
    eje.grid(axis="y", linestyle="--", alpha=0.4)
    for lado in ("top", "right"):
        eje.spines[lado].set_visible(False)
    return guardar_figura(figura, "temperatura_semanal.png")
```

SALIDA

```
Gráfico generado: salidas/graficos/temperatura_semanal.png
```

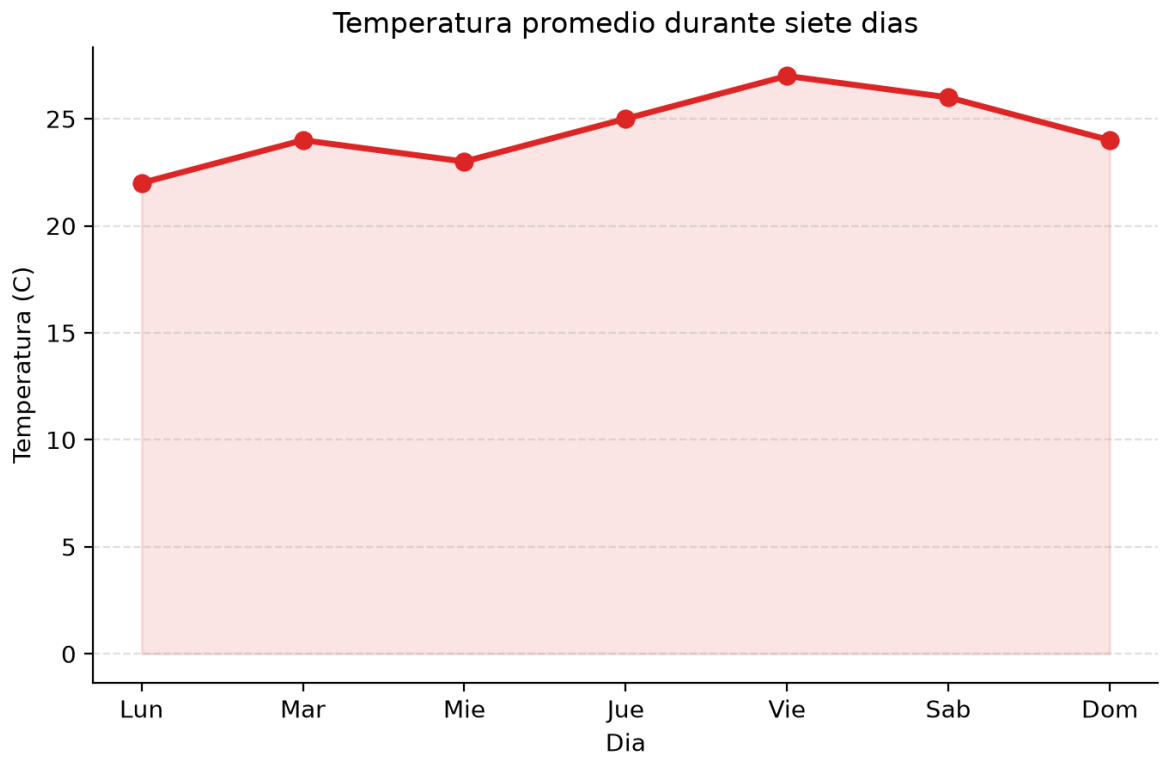


Figura 2. Temperatura promedio durante siete días.

8.3 Gráfico circular de preferencias

ENUNCIADO

Crea un gráfico circular con las preferencias de fruta de un grupo.

SOLUCIÓN

El gráfico muestra la participación porcentual de cada fruta. Se separa ligeramente el sector de mayor tamaño para destacarlo.

CÓDIGO

```
def grafico_circular_frutas() -> Path:
    frutas = ["Manzana", "Banana", "Naranja", "Uva", "Pera"]
    cantidades = [35, 25, 20, 12, 8]
    colores = ["#EF4444", "#F59E0B", "#FB923C", "#8B5CF6", "#84CC16"]
    figura, eje = plt.subplots(figsize=(6.8, 5.6))
    eje.pie(
        cantidades,
        labels=frutas,
        colors=colores,
        autopct="%1.1f%%",
        startangle=90,
        explode=[0.04] * len(frutas),
        wedgeprops={"edgecolor": "white", "linewidth": 2},
    )
    eje.set_title("Preferencias de fruta del grupo")
    eje.axis("equal")
    return guardar_figura(figura, "preferencias_frutas.png")
```

SALIDA

```
Gráfico generado: salidas/graficos/preferencias_frutas.png
```

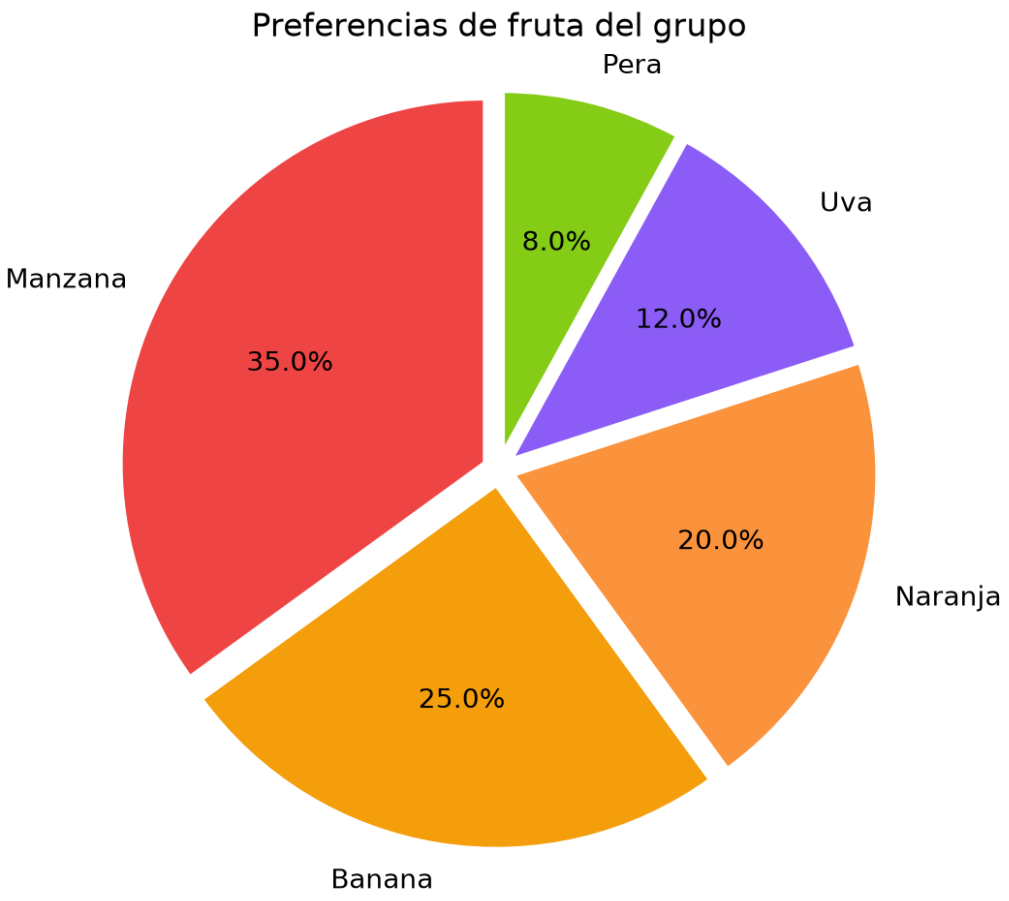


Figura 3. Preferencias de fruta del grupo.

9. Programación orientada a objetos

Se modelan objetos con atributos y métodos, se aplica encapsulamiento y se reutiliza comportamiento mediante herencia.

9.1 Clase Coche

ENUNCIADO

Crea una clase Coche con atributos marca y modelo, y un método que los muestre.

SOLUCIÓN

El constructor almacena marca y modelo. El método mostrar devuelve una cadena formateada, lo que separa la representación del objeto de la impresión en consola.

CÓDIGO

```
class Coche:
    def __init__(self, marca: str, modelo: str) -> None:
        self.marca = marca
        self.modelo = modelo

    def mostrar(self) -> str:
        return f"Coche: {self.marca} {self.modelo}"
```

SALIDA

```
Coche: Toyota Corolla
```

9.2 Clase Persona y mayoría de edad

ENUNCIADO

Crea una clase Persona con un método que indique si es mayor de edad.

SOLUCIÓN

La clase almacena nombre y edad. El método es_mayor_de_edad compara la edad con 18 y describir utiliza ese resultado para generar un mensaje legible.

CÓDIGO

```
class Persona:
    def __init__(self, nombre: str, edad: int) -> None:
        self.nombre = nombre
        self.edad = edad

    def es_mayor_de_edad(self) -> bool:
        return self.edad >= 18

    def describir(self) -> str:
        estado = "mayor de edad" if self.es_mayor_de_edad() else "menor de edad"
        return f"{self.nombre} tiene {self.edad} anos y es {estado}"
```

SALIDA

```
Ana tiene 20 anos y es mayor de edad
```

9.3 Herencia Animal y Perro

ENUNCIADO

Implementa herencia creando una clase Animal y una subclase Perro.

SOLUCIÓN

Animal define el nombre y un sonido genérico. Perro hereda el comportamiento y sobrescribe hablar para devolver su sonido característico.

CÓDIGO

```
class Animal:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre

    def hablar(self) -> str:
        return "Sonido generico"

    def presentarse(self) -> str:
        return f"{self.nombre}: {self.hablar()}"

class Perro(Animal):
    def hablar(self) -> str:
        return "Guau guau"
```

SALIDA

```
Firulais: Guau guau
```

10. Manejo de errores y excepciones

10.1 Captura de IndexError

ENUNCIADO

Crea un programa que capture un error de índice fuera de rango.

SOLUCIÓN

La función intenta acceder a la posición solicitada dentro de una secuencia. Si el índice no existe, captura IndexError, informa el problema y devuelve None.

CÓDIGO

```
def obtener_elemento(secuencia: Sequence[object], indice: int) -> object | None:
    try:
        return secuencia[indice]
    except IndexError:
        print("Error capturado: indice fuera de rango.")
        return None
```

SALIDA

```
Error capturado: indice fuera de rango.
Elemento en índice 5: None
```

10.2 Excepción personalizada para edades

ENUNCIADO

Crea una excepción personalizada para validar que una edad no sea negativa.

SOLUCIÓN

EdadInvalidaError hereda de ValueError. La función validar_edad lanza la excepción cuando recibe un valor negativo y el programa principal la captura para mostrar un mensaje controlado.

CÓDIGO

```
class EdadInvalidaError(ValueError):
    pass

def validar_edad(edad: int) -> int:
    if edad < 0:
        raise EdadInvalidaError("La edad no puede ser negativa.")
    return edad
```

SALIDA

```
Excepción personalizada: La edad no puede ser negativa.
```

11. Manejo de archivos

11.1 Guardar y leer nombres

ENUNCIADO

Escribe un programa que guarde nombres en un archivo y luego los lea.

SOLUCIÓN

Se utiliza la codificación UTF-8 y la sentencia with para cerrar el archivo automáticamente. La lectura descarta líneas vacías y espacios sobrantes.

CÓDIGO

```
def guardar_nombres(ruta: Path, nombres: Iterable[str]) -> None:
    ruta.parent.mkdir(parents=True, exist_ok=True)
    with ruta.open("w", encoding="utf-8") as archivo:
        for nombre in nombres:
            archivo.write(f"{nombre}\n")

def leer_nombres(ruta: Path) -> list[str]:
    with ruta.open("r", encoding="utf-8") as archivo:
        return [linea.strip() for linea in archivo if linea.strip()]
```

SALIDA

```
Nombres leídos: ['Ana', 'Luis', 'Maria']
```

11.2 Lectura de un CSV con Pandas

ENUNCIADO

Usa pandas para leer un archivo CSV y mostrar los primeros 5 registros.

SOLUCIÓN

El DataFrame se exporta a CSV y se vuelve a leer con read_csv. El método head(5) limita la salida a las primeras cinco filas.

CÓDIGO

```
def guardar_csv_estudiantes(dataframe: pd.DataFrame, ruta: Path) -> None:
    ruta.parent.mkdir(parents=True, exist_ok=True)
    dataframe.to_csv(ruta, index=False, encoding="utf-8")

def leer_primeros_registros(ruta: Path, cantidad: int = 5) -> pd.DataFrame:
    return pd.read_csv(ruta, encoding="utf-8").head(cantidad)
```

SALIDA

```
Estudiante  Nota 1  Nota 2  Nota 3  Promedio
       Ana      85      90      78     84.33
      Luis      62      70      55     62.33
     Maria      91      87      95     91.00
    Carlos      74      68      80     74.00
     Sofia      58      65      72     65.00
```

12. Programación funcional

12.1 Filtrado de números pares

ENUNCIADO

Usa filter para obtener solo los números pares de una lista.

SOLUCIÓN

La función recibe un iterable y aplica una función lambda como criterio de filtrado. El resultado se convierte a lista para poder imprimirlo y reutilizarlo.

CÓDIGO

```
def filtrar_pares(numeros: Iterable[int]) -> list[int]:
    return list(filter(lambda numero: numero % 2 == 0, numeros))
```

SALIDA

```
[2, 4, 6]
```

12.2 Producto con reduce

ENUNCIADO

Usa reduce para multiplicar todos los elementos de una lista.

SOLUCIÓN

reduce aplica una función de dos argumentos de forma acumulativa. El valor inicial 1 garantiza el resultado correcto incluso para listas vacías.

CÓDIGO

```
def multiplicar_elementos(numeros: Iterable[float]) -> float:
    return reduce(lambda acumulado, numero: acumulado * numero, numeros, 1)
```

SALIDA

```
720
```

13. Programación asíncrona

Las tareas asíncronas permiten iniciar varias operaciones y continuar con otras mientras se espera. En los ejemplos, asyncio.sleep simula el tiempo de espera de una operación de entrada/salida.

13.1 Dos tareas asíncronas en paralelo

ENUNCIADO

Crea dos tareas asíncronas que impriman mensajes en paralelo.

SOLUCIÓN

asyncio.gather programa ambas corrutinas y espera a que terminen. Los mensajes de inicio aparecen juntos y los mensajes de fin se imprimen al completarse cada tarea.

CÓDIGO

```
async def tarea_asincrona(nombre: str, duracion: float) -> str:
    print(f"{nombre}: inicio")
    await asyncio.sleep(duracion)
    print(f"{nombre}: fin")
    return nombre

async def ejecutar_tareas_en_paralelo() -> list[str]:
    return await asyncio.gather(
        tarea_asincrona("Tarea A", 1.0),
        tarea_asincrona("Tarea B", 1.0),
    )
```

SALIDA

```
Tarea A: inicio
Tarea B: inicio
Tarea A: fin
Tarea B: fin
['Tarea A', 'Tarea B']
```

13.2 Simulación de descargas

ENUNCIADO

Simula la descarga de archivos con asyncio.sleep.

SOLUCIÓN

Tres descargas de distinta duración se ejecutan de forma concurrente. El tiempo total se aproxima a la descarga más lenta, no a la suma de todas las duraciones.

CÓDIGO

```
async def descargar_archivo(nombre: str, duracion: float) -> str:
    print(f"Descargando {nombre}...")
    await asyncio.sleep(duracion)
    print(f"{nombre} descargado.")
    return nombre

async def simular_descargas() -> list[str]:
    inicio = time.perf_counter()
    resultados = await asyncio.gather(
        descargar_archivo("documento.pdf", 1.2),
        descargar_archivo("imagen.png", 0.8),
        descargar_archivo("datos.csv", 1.0),
    )
    print(f"Tiempo total de descarga: {time.perf_counter() - inicio:.2f} segundos")
    return resultados
```

SALIDA

```
Descargando documento.pdf...
Descargando imagen.png...
```

```
Descargando datos.csv...
imagen.png descargado.
datos.csv descargado.
documento.pdf descargado.
Tiempo total de descarga: 1.21 segundos
['documento.pdf', 'imagen.png', 'datos.csv']
```

14. Testing con unittest

Las pruebas unitarias comprueban el comportamiento esperado de funciones pequeñas. Se cubren casos normales, valores negativos, decimales, listas desordenadas, duplicados y listas vacías.

14.1 Pruebas para la función restar

ENUNCIADO

Crea pruebas unitarias para una función que reste dos números.

SOLUCIÓN

Se verifican una resta positiva, una resta con números negativos y una resta con decimales. assertEqual compara valores exactos y assertAlmostEqual compara decimales con tolerancia.

CÓDIGO

```
def restar(a: float, b: float) -> float:
    return a - b

class TestResta(unittest.TestCase):
    def test_resta_positiva(self) -> None:
        self.assertEqual(restar(10, 4), 6)

    def test_resta_con_negativos(self) -> None:
        self.assertEqual(restar(-5, -3), -2)

    def test_resta_decimal(self) -> None:
        self.assertAlmostEqual(restar(7.5, 2.25), 5.25)
```

SALIDA

```
test_resta_con_negativos (test_soluciones.TestResta.test_resta_con_negativos) ... ok
test_resta_decimal (test_soluciones.TestResta.test_resta_decimal) ... ok
test_resta_positiva (test_soluciones.TestResta.test_resta_positiva) ... ok

----------------------------------------------------------------------
Ran 3 tests in 0.000s

OK
```

14.2 Pruebas para verificar una lista ordenada

ENUNCIADO

Implementa pruebas para verificar que una lista está ordenada.

SOLUCIÓN

Se comprueban listas ordenadas, desordenadas, con duplicados y vacías. La función esta_ordenada recorre pares consecutivos y verifica que cada elemento sea menor o igual que el siguiente.

CÓDIGO

```
def esta_ordenada(numeros: Sequence[float]) -> bool:
    return all(
        numeros[indice] <= numeros[indice + 1]
        for indice in range(len(numeros) - 1)
    )

class TestListaOrdenada(unittest.TestCase):
    def test_lista_ordenada(self) -> None:
```

```
        self.assertTrue(esta_ordenada([1, 2, 3, 4]))

    def test_lista_desordenada(self) -> None:
        self.assertFalse(esta_ordenada([3, 1, 2]))

    def test_lista_con_duplicados_ordenados(self) -> None:
        self.assertTrue(esta_ordenada([1, 1, 2, 2, 3]))

    def test_lista_vacia(self) -> None:
        self.assertTrue(esta_ordenada([]))
```

SALIDA

```
test_lista_con_duplicados_ordenados
(test_soluciones.TestListaOrdenada.test_lista_con_duplicados_ordenados) ... ok
test_lista_desordenada (test_soluciones.TestListaOrdenada.test_lista_desordenada) ... ok
test_lista_ordenada (test_soluciones.TestListaOrdenada.test_lista_ordenada) ... ok
test_lista_vacia (test_soluciones.TestListaOrdenada.test_lista_vacia) ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```

15. Conclusiones

La práctica permitió recorrer los fundamentos de Python desde las operaciones básicas hasta la programación asíncrona y las pruebas unitarias. La separación entre funciones de cálculo, clases y menú facilita probar cada solución y reutilizarla. Los ejercicios de Pandas y visualización muestran cómo transformar datos en información presentable, mientras que unittest aporta una verificación automática del comportamiento esperado. El código entregado no contiene comentarios ni marcas de edición, y puede ejecutarse directamente en Visual Studio o Visual Studio Code.

Anexo A. Código completo

Se incluye el código completo de los tres archivos que forman la solución. Los bloques se presentan con una fuente monoespaciada y sin comentarios.

soluciones_fundamentos.py

```
import asyncio
import math
import sys
import time
from functools import reduce
from pathlib import Path
from typing import Iterable, Sequence

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import pandas as pd
from matplotlib.figure import Figure

from utilidades import millas_a_kilometros

BASE_DIR = Path(__file__).resolve().parent
SALIDAS_DIR = BASE_DIR / "salidas"
GRAFICOS_DIR = SALIDAS_DIR / "graficos"


def solicitar_float(mensaje: str) -> float:
    while True:
        try:
            return float(input(mensaje).replace(",", "."))
        except ValueError:
            print("Entrada invalida. Ingrese un numero.")


def solicitar_entero(mensaje: str) -> int:
    while True:
        try:
            return int(input(mensaje))
        except ValueError:
            print("Entrada invalida. Ingrese un numero entero.")


def solicitar_lista_numeros(mensaje: str) -> list[float]:
    texto = input(mensaje)
    return [float(parte.strip()) for parte in texto.split(",") if parte.strip()]


def promedio_tres_numeros(a: float, b: float, c: float) -> float:
    return (a + b + c) / 3


def celsius_a_fahrenheit(celsius: float) -> float:
    return celsius * 9 / 5 + 32


def area_triangulo(base: float, altura: float) -> float:
    return base * altura / 2


def puede_votar(edad: int) -> bool:
    if edad < 0:
        raise ValueError("La edad no puede ser negativa.")
    return edad >= 18


def tabla_multiplicar(numero: int, limite: int = 10) -> list[str]:
    return [f"{numero} x {valor} = {numero * valor}" for valor in range(1, limite + 1)]


def menu_calculadora() -> None:
    while True:
        print("\n1. Sumar")
        print("2. Restar")
        print("3. Salir")
        opcion = input("Seleccione una opcion: ").strip()
        if opcion == "3":
            print("Calculadora finalizada.")
            return
```

```
        if opcion not in {"1", "2"}:
            print("Opcion invalida.")
            continue
        primer_numero = solicitar_float("Primer numero: ")
        segundo_numero = solicitar_float("Segundo numero: ")
        resultado = (
            primer_numero + segundo_numero
            if opcion == "1"
            else primer_numero - segundo_numero
        )
        print(f"Resultado: {resultado:.2f}")


def factorial(numero: int) -> int:
    if numero < 0:
        raise ValueError("El factorial no esta definido para numeros negativos.")
    resultado = 1
    for valor in range(2, numero + 1):
        resultado *= valor
    return resultado


def es_primo(numero: int) -> bool:
    if numero < 2:
        return False
    if numero == 2:
        return True
    if numero % 2 == 0:
        return False
    limite = math.isqrt(numero)
    for divisor in range(3, limite + 1, 2):
        if numero % divisor == 0:
            return False
    return True


def mayor_lista(numeros: Sequence[float]) -> float:
    if not numeros:
        raise ValueError("La lista no puede estar vacia.")
    return max(numeros)


def mayores_de_edad(personas: dict[str, int]) -> dict[str, int]:
    return {
        nombre: edad
        for nombre, edad in personas.items()
        if edad >= 18
    }


def ordenar_numeros(numeros: Iterable[float]) -> list[float]:
    return sorted(numeros)


def eliminar_duplicados(elementos: Iterable[object]) -> set[object]:
    return set(elementos)


def crear_dataframe_estudiantes() -> pd.DataFrame:
    datos = {
        "Estudiante": ["Ana", "Luis", "Maria", "Carlos", "Sofia", "Diego"],
        "Nota 1": [85, 62, 91, 74, 58, 88],
        "Nota 2": [90, 70, 87, 68, 65, 92],
        "Nota 3": [78, 55, 95, 80, 72, 85],
    }
    dataframe = pd.DataFrame(datos)
    dataframe["Promedio"] = (
        dataframe[["Nota 1", "Nota 2", "Nota 3"]]
        .mean(axis=1)
        .round(2)
    )
    return dataframe


def filtrar_estudiantes_aprobados(
    dataframe: pd.DataFrame, umbral: float = 70
) -> pd.DataFrame:
    return dataframe[dataframe["Promedio"] > umbral].copy()


def agregar_estado(dataframe: pd.DataFrame, umbral: float = 70) -> pd.DataFrame:
    resultado = dataframe.copy()
    resultado["Estado"] = resultado["Promedio"].apply(
        lambda nota: "Aprobado" if nota >= umbral else "Reprobado"
    )
    return resultado
```

```
def guardar_csv_estudiantes(dataframe: pd.DataFrame, ruta: Path) -> None:
    ruta.parent.mkdir(parents=True, exist_ok=True)
    dataframe.to_csv(ruta, index=False, encoding="utf-8")


def leer_primeros_registros(ruta: Path, cantidad: int = 5) -> pd.DataFrame:
    return pd.read_csv(ruta, encoding="utf-8").head(cantidad)


def guardar_figura(figura: Figure, nombre: str) -> Path:
    GRAFICOS_DIR.mkdir(parents=True, exist_ok=True)
    ruta = GRAFICOS_DIR / nombre
    figura.savefig(ruta, dpi=200, bbox_inches="tight")
    plt.close(figura)
    return ruta


def grafico_barras_ventas() -> Path:
    meses = ["Ene", "Feb", "Mar", "Abr", "May", "Jun"]
    ventas = [1200, 1500, 1800, 1650, 2100, 2400]
    figura, eje = plt.subplots(figsize=(8, 4.8))
    barras = eje.bar(meses, ventas, color="#2563EB", edgecolor="#1E3A8A")
    eje.set_title("Ventas mensuales del negocio")
    eje.set_xlabel("Mes")
    eje.set_ylabel("Ventas (USD)")
    eje.grid(axis="y", linestyle="--", alpha=0.4)
    for lado in ("top", "right"):
        eje.spines[lado].set_visible(False)
    eje.bar_label(barras, fmt="%d", padding=3)
    return guardar_figura(figura, "ventas_mensuales.png")


def grafico_lineas_temperatura() -> Path:
    dias = ["Lun", "Mar", "Mie", "Jue", "Vie", "Sab", "Dom"]
    temperaturas = [22, 24, 23, 25, 27, 26, 24]
    figura, eje = plt.subplots(figsize=(8, 4.8))
    eje.plot(
        dias,
        temperaturas,
        marker="o",
        markersize=7,
        linewidth=2.5,
        color="#DC2626",
    )
    eje.fill_between(dias, temperaturas, color="#DC2626", alpha=0.12)
    eje.set_title("Temperatura promedio durante siete dias")
    eje.set_xlabel("Dia")
    eje.set_ylabel("Temperatura (C)")
    eje.grid(axis="y", linestyle="--", alpha=0.4)
    for lado in ("top", "right"):
        eje.spines[lado].set_visible(False)
    return guardar_figura(figura, "temperatura_semanal.png")


def grafico_circular_frutas() -> Path:
    frutas = ["Manzana", "Banana", "Naranja", "Uva", "Pera"]
    cantidades = [35, 25, 20, 12, 8]
    colores = ["#EF4444", "#F59E0B", "#FB923C", "#8B5CF6", "#84CC16"]
    figura, eje = plt.subplots(figsize=(6.8, 5.6))
    eje.pie(
        cantidades,
        labels=frutas,
        colors=colores,
        autopct="%1.1f%%",
        startangle=90,
        explode=[0.04] * len(frutas),
        wedgeprops={"edgecolor": "white", "linewidth": 2},
    )
    eje.set_title("Preferencias de fruta del grupo")
    eje.axis("equal")
    return guardar_figura(figura, "preferencias_frutas.png")


class Coche:
    def __init__(self, marca: str, modelo: str) -> None:
        self.marca = marca
        self.modelo = modelo

    def mostrar(self) -> str:
        return f"Coche: {self.marca} {self.modelo}"


class Persona:
    def __init__(self, nombre: str, edad: int) -> None:
        self.nombre = nombre
        self.edad = edad

    def es_mayor_de_edad(self) -> bool:
```

```
        return self.edad >= 18

    def describir(self) -> str:
        estado = "mayor de edad" if self.es_mayor_de_edad() else "menor de edad"
        return f"{self.nombre} tiene {self.edad} anos y es {estado}"


class Animal:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre

    def hablar(self) -> str:
        return "Sonido generico"

    def presentarse(self) -> str:
        return f"{self.nombre}: {self.hablar()}"


class Perro(Animal):
    def hablar(self) -> str:
        return "Guau guau"


class EdadInvalidaError(ValueError):
    pass


def validar_edad(edad: int) -> int:
    if edad < 0:
        raise EdadInvalidaError("La edad no puede ser negativa.")
    return edad


def obtener_elemento(secuencia: Sequence[object], indice: int) -> object | None:
    try:
        return secuencia[indice]
    except IndexError:
        print("Error capturado: indice fuera de rango.")
        return None


def guardar_nombres(ruta: Path, nombres: Iterable[str]) -> None:
    ruta.parent.mkdir(parents=True, exist_ok=True)
    with ruta.open("w", encoding="utf-8") as archivo:
        for nombre in nombres:
            archivo.write(f"{nombre}\n")


def leer_nombres(ruta: Path) -> list[str]:
    with ruta.open("r", encoding="utf-8") as archivo:
        return [linea.strip() for linea in archivo if linea.strip()]


def filtrar_pares(numeros: Iterable[int]) -> list[int]:
    return list(filter(lambda numero: numero % 2 == 0, numeros))


def multiplicar_elementos(numeros: Iterable[float]) -> float:
    return reduce(lambda acumulado, numero: acumulado * numero, numeros, 1)


async def tarea_asincrona(nombre: str, duracion: float) -> str:
    print(f"{nombre}: inicio")
    await asyncio.sleep(duracion)
    print(f"{nombre}: fin")
    return nombre


async def ejecutar_tareas_en_paralelo() -> list[str]:
    return await asyncio.gather(
        tarea_asincrona("Tarea A", 1.0),
        tarea_asincrona("Tarea B", 1.0),
    )


async def descargar_archivo(nombre: str, duracion: float) -> str:
    print(f"Descargando {nombre}...")
    await asyncio.sleep(duracion)
    print(f"{nombre} descargado.")
    return nombre


async def simular_descargas() -> list[str]:
    inicio = time.perf_counter()
    resultados = await asyncio.gather(
        descargar_archivo("documento.pdf", 1.2),
        descargar_archivo("imagen.png", 0.8),
        descargar_archivo("datos.csv", 1.0),
```

```
    )
    print(f"Tiempo total de descarga: {time.perf_counter() - inicio:.2f} segundos")
    return resultados


def restar(a: float, b: float) -> float:
    return a - b


def esta_ordenada(numeros: Sequence[float]) -> bool:
    return all(
        numeros[indice] <= numeros[indice + 1]
        for indice in range(len(numeros) - 1)
    )


def mostrar_titulo(titulo: str) -> None:
    print("\n" + "=" * 72)
    print(titulo)
    print("=" * 72)


def ejecutar_demo() -> None:
    mostrar_titulo("FUNDAMENTOS DE PYTHON - DEMOSTRACION DE SOLUCIONES")

    print("\n[1] Promedio de tres numeros")
    print(f"Promedio de 8, 9 y 10: {promedio_tres_numeros(8, 9, 10):.2f}")

    print("\n[2] Conversion de Celsius a Fahrenheit")
    print(f"25 C = {celsius_a_fahrenheit(25):.2f} F")

    print("\n[3] Area de un triangulo")
    print(f"Base 10 y altura 6: {area_triangulo(10, 6):.2f}")

    print("\n[4] Validacion para votar")
    for edad in (16, 18, 30):
        print(f"Edad {edad}: {'puede votar' if puede_votar(edad) else 'no puede votar'}")

    print("\n[5] Tabla de multiplicar del 7")
    for linea in tabla_multiplicar(7):
        print(linea)

    print("\n[6] Menu de calculadora")
    print("La calculadora interactiva se ejecuta con la opcion 6 del menu principal.")

    print("\n[7] Factorial")
    for numero in (0, 5, 7):
        print(f"{numero}! = {factorial(numero)}")

    print("\n[8] Numeros primos")
    print(f"17 es primo: {es_primo(17)}")
    print(f"21 es primo: {es_primo(21)}")

    print("\n[9] Mayor de una lista")
    print(f"Mayor de [12, 45, 7, 31]: {mayor_lista([12, 45, 7, 31])}")

    print("\n[10] Personas mayores de edad")
    personas = {"Ana": 17, "Luis": 25, "Maria": 30, "Pedro": 15}
    print(mayores_de_edad(personas))

    print("\n[11] Ordenamiento de una lista")
    print(ordenar_numeros([9, 3, 7, 1, 5]))

    print("\n[12] Eliminacion de duplicados")
    print(sorted(eliminar_duplicados([1, 2, 2, 3, 4, 4, 5])))

    print("\n[13] Modulo utilidades")
    print(f"10 millas = {millas_a_kilometros(10):.2f} kilometros")

    print("\n[14] DataFrame de estudiantes")
    estudiantes = crear_dataframe_estudiantes()
    print(estudiantes.to_string(index=False))
    print(f"Promedio general del curso: {estudiantes['Promedio'].mean():.2f}")

    print("\n[15] Estudiantes con promedio mayor a 70")
    aprobados = filtrar_estudiantes_aprobados(estudiantes)
    print(aprobados[["Estudiante", "Promedio"]].to_string(index=False))

    print("\n[16] Estado academico")
    print(agregar_estado(estudiantes).to_string(index=False))

    ruta_csv = SALIDAS_DIR / "estudiantes.csv"
    guardar_csv_estudiantes(estudiantes, ruta_csv)

    print("\n[17] Lectura de las primeras cinco filas del CSV")
    print(leer_primeros_registros(ruta_csv).to_string(index=False))

    print("\n[18] Graficos con matplotlib")
```

```
    rutas = [
        grafico_barras_ventas(),
        grafico_lineas_temperatura(),
        grafico_circular_frutas(),
    ]
    for ruta in rutas:
        print(f"Grafico generado: {ruta.relative_to(BASE_DIR)}")

    print("\n[19] Programacion orientada a objetos")
    coche = Coche("Toyota", "Corolla")
    persona = Persona("Ana", 20)
    perro = Perro("Firulais")
    print(coche.mostrar())
    print(persona.describir())
    print(perro.presentarse())

    print("\n[20] Manejo de excepciones")
    print(f"Elemento en indice 5: {obtener_elemento([10, 20, 30], 5)}")
    try:
        validar_edad(-3)
    except EdadInvalidaError as error:
        print(f"Excepcion personalizada: {error}")


    print("\n[21] Manejo de archivos")
    ruta_nombres = SALIDAS_DIR / "nombres.txt"
    guardar_nombres(ruta_nombres, ["Ana", "Luis", "Maria"])
    print(f"Nombres leidos: {leer_nombres(ruta_nombres)}")

    print("\n[22] Programacion funcional")
    numeros = [1, 2, 3, 4, 5, 6]
    print(f"Pares: {filtrar_pares(numeros)}")
    print(f"Producto: {multiplicar_elementos(numeros)}")

    print("\n[23] Programacion asincrona")
    tareas = asyncio.run(ejecutar_tareas_en_paralelo())
    print(f"Tareas completadas: {tareas}")
    descargas = asyncio.run(simular_descargas())
    print(f"Descargas completadas: {descargas}")

    print("\n[24] Funciones para pruebas unitarias")
    print(f"restar(10, 4) = {restar(10, 4)}")
    print(f"esta_ordenada([1, 2, 3]) = {esta_ordenada([1, 2, 3])}")
    print(f"esta_ordenada([3, 1, 2]) = {esta_ordenada([3, 1, 2])}")

    print("\nDemostracion finalizada correctamente.")


def interactivo_promedio() -> None:
    valores = [solicitar_float(f"Numero {indice}: ") for indice in range(1, 4)]
    print(f"Promedio: {promedio_tres_numeros(*valores):.2f}")


def interactivo_celsius() -> None:
    celsius = solicitar_float("Temperatura en Celsius: ")
    print(f"Temperatura en Fahrenheit: {celsius_a_fahrenheit(celsius):.2f}")


def interactivo_area() -> None:
    base = solicitar_float("Base: ")
    altura = solicitar_float("Altura: ")
    print(f"Area: {area_triangulo(base, altura):.2f}")


def interactivo_votar() -> None:
    edad = solicitar_entero("Edad: ")
    print("Puede votar." if puede_votar(edad) else "No puede votar.")


def interactivo_tabla() -> None:
    numero = solicitar_entero("Numero: ")
    for linea in tabla_multiplicar(numero):
        print(linea)


def interactivo_factorial() -> None:
    numero = solicitar_entero("Numero: ")
    print(f"Factorial: {factorial(numero)}")


def interactivo_primo() -> None:
    numero = solicitar_entero("Numero: ")
    print("Es primo." if es_primo(numero) else "No es primo.")


def interactivo_mayor() -> None:
    numeros = solicitar_lista_numeros("Numeros separados por coma: ")
    print(f"Mayor: {mayor_lista(numeros)}")
```

```
def interactivo_colecciones() -> None:
    personas = {}
    for indice in range(1, 4):
        nombre = input(f"Nombre {indice}: ").strip()
        personas[nombre] = solicitar_entero(f"Edad de {nombre}: ")
    print(f"Mayores de edad: {mayores_de_edad(personas)}")
    numeros = solicitar_lista_numeros("Numeros separados por coma: ")
    print(f"Ordenados: {ordenar_numeros(numeros)}")
    print(f"Sin duplicados: {sorted(eliminar_duplicados(numeros))}")


def interactivo_millas() -> None:
    millas = solicitar_float("Millas: ")
    print(f"Kilometros: {millas_a_kilometros(millas):.3f}")


def interactivo_pandas() -> None:
    estudiantes = crear_dataframe_estudiantes()
    print(estudiantes.to_string(index=False))
    print(f"Promedio general: {estudiantes['Promedio'].mean():.2f}")
    print("\nPromedio mayor a 70:")
    print(filtrar_estudiantes_aprobados(estudiantes).to_string(index=False))
    print("\nEstado academico:")
    print(agregar_estado(estudiantes).to_string(index=False))


def interactivo_graficos() -> None:
    rutas = [
        grafico_barras_ventas(),
        grafico_lineas_temperatura(),
        grafico_circular_frutas(),
    ]
    for ruta in rutas:
        print(f"Grafico generado: {ruta}")


def interactivo_poo() -> None:
    coche = Coche("Honda", "Civic")
    persona = Persona("Carlos", 17)
    perro = Perro("Rex")
    print(coche.mostrar())
    print(persona.describir())
    print(perro.presentarse())


def interactivo_excepciones() -> None:
    indice = solicitar_entero("Indice a consultar: ")
    print(f"Resultado: {obtener_elemento([10, 20, 30], indice)}")
    try:
        edad = solicitar_entero("Edad para validar: ")
        validar_edad(edad)
        print("Edad valida.")
    except EdadInvalidaError as error:
        print(f"Error: {error}")


def interactivo_archivos() -> None:
    ruta = SALIDAS_DIR / "nombres.txt"
    nombres = []
    for indice in range(1, 4):
        nombres.append(input(f"Nombre {indice}: ").strip())
    guardar_nombres(ruta, nombres)
    print(f"Nombres guardados y leidos: {leer_nombres(ruta)}")


def interactivo_csv() -> None:
    estudiantes = crear_dataframe_estudiantes()
    ruta = SALIDAS_DIR / "estudiantes.csv"
    guardar_csv_estudiantes(estudiantes, ruta)
    print(leer_primeros_registros(ruta).to_string(index=False))


def interactivo_funcional() -> None:
    numeros = solicitar_lista_numeros("Numeros separados por coma: ")
    enteros = [int(numero) for numero in numeros]
    print(f"Pares: {filtrar_pares(enteros)}")
    print(f"Producto: {multiplicar_elementos(numeros)}")


def interactivo_async() -> None:
    print(asyncio.run(ejecutar_tareas_en_paralelo()))
    print(asyncio.run(simular_descargas()))


def interactivo_testing() -> None:
    import unittest
```

```
    from test_soluciones import TestListaOrdenada, TestResta

    suite = unittest.TestSuite()
    suite.addTests(unittest.defaultTestLoader.loadTestsFromTestCase(TestResta))
    suite.addTests(
        unittest.defaultTestLoader.loadTestsFromTestCase(TestListaOrdenada)
    )
    unittest.TextTestRunner(verbosity=2).run(suite)


OPCIONES = [
    ("Promedio de tres numeros", interactivo_promedio),
    ("Conversion de Celsius a Fahrenheit", interactivo_celsius),
    ("Area de un triangulo", interactivo_area),
    ("Validacion para votar", interactivo_votar),
    ("Tabla de multiplicar", interactivo_tabla),
    ("Menu de calculadora", menu_calculadora),
    ("Factorial de un numero", interactivo_factorial),
    ("Verificacion de numero primo", interactivo_primo),
    ("Mayor de una lista", interactivo_mayor),
    ("Colecciones y duplicados", interactivo_colecciones),
    ("Conversion de millas a kilometros", interactivo_millas),
    ("Ejercicios con Pandas", interactivo_pandas),
    ("Graficos con matplotlib", interactivo_graficos),
    ("Programacion orientada a objetos", interactivo_poo),
    ("Manejo de excepciones", interactivo_excepciones),
    ("Manejo de archivos", interactivo_archivos),
    ("Lectura de CSV con Pandas", interactivo_csv),
    ("Programacion funcional", interactivo_funcional),
    ("Programacion asincrona", interactivo_async),
    ("Pruebas unitarias", interactivo_testing),
]


def menu_principal() -> None:
    while True:
        print("\n" + "=" * 72)
        print("FUNDAMENTOS DE PYTHON - MENU DE EJERCICIOS")
        print("=" * 72)
        for indice, (titulo, _) in enumerate(OPCIONES, start=1):
            print(f"{indice:2d}. {titulo}")
        print(" 0. Salir")
        opcion = input("Seleccione una opcion: ").strip()
        if opcion == "0":
            print("Programa finalizado.")
            return
        if not opcion.isdigit():
            print("Opcion invalida.")
            continue
        indice = int(opcion)
        if not 1 <= indice <= len(OPCIONES):
            print("Opcion invalida.")
            continue
        try:
            OPCIONES[indice - 1][1]()
        except (ValueError, TypeError) as error:
            print(f"Error: {error}")


def main() -> None:
    if "--demo" in sys.argv:
        ejecutar_demo()
        return
    menu_principal()


if __name__ == "__main__":
    main()
```

utilidades.py

```
def millas_a_kilometros(millas: float) -> float:
    return millas * 1.609344
```

test_soluciones.py

```
import unittest

from soluciones_fundamentos import esta_ordenada, restar


class TestResta(unittest.TestCase):
```

```
    def test_resta_positiva(self) -> None:
        self.assertEqual(restar(10, 4), 6)

    def test_resta_con_negativos(self) -> None:
        self.assertEqual(restar(-5, -3), -2)

    def test_resta_decimal(self) -> None:
        self.assertAlmostEqual(restar(7.5, 2.25), 5.25)


class TestListaOrdenada(unittest.TestCase):
    def test_lista_ordenada(self) -> None:
        self.assertTrue(esta_ordenada([1, 2, 3, 4]))

    def test_lista_desordenada(self) -> None:
        self.assertFalse(esta_ordenada([3, 1, 2]))

    def test_lista_con_duplicados_ordenados(self) -> None:
        self.assertTrue(esta_ordenada([1, 1, 2, 2, 3]))

    def test_lista_vacia(self) -> None:
        self.assertTrue(esta_ordenada([]))


if __name__ == "__main__":
    unittest.main()
```

Anexo B. Ejecución en Visual Studio y VS Code

La carpeta de la práctica puede abrirse directamente como proyecto. Los pasos siguientes preparan el entorno y ejecutan la solución.

1. Abrir la carpeta 01_Fundamentos_Python en Visual Studio (Archivo > Abrir > Carpeta) o en VS Code (File > Open Folder).
2. Crear el entorno virtual con py -m venv .venv.
3. Activar el entorno en PowerShell con .\.venv\Scripts\Activate.ps1, o en CMD con .\.venv\Scripts\activate.bat.
4. Instalar las dependencias con py -m pip install -r requirements.txt.
5. Ejecutar la demostración completa con py soluciones_fundamentos.py --demo.
6. Ejecutar el menú interactivo con py soluciones_fundamentos.py.
7. Ejecutar las pruebas con py -m unittest -v.

En Visual Studio, seleccionar el intérprete de Python del entorno virtual y presionar F5 sobre soluciones_fundamentos.py. En VS Code, usar Run Python File. Las salidas se guardan en la carpeta salidas y los gráficos en salidas/graficos.

Elemento	Detalle
Programa principal	soluciones_fundamentos.py
Demostración	py soluciones_fundamentos.py --demo
Menú interactivo	py soluciones_fundamentos.py
Pruebas	py -m unittest -v
Salidas	salidas/
Gráficos	salidas/graficos/